

# Backend Basics

# Inhaltsverzeichnis

1. MVC
2. REST API
3. RESTful Principles
4. Idempotenz
5. API Dokumentation
6. API Versionierung
7. Business Logic
8. Databases Fundamentals
9. Query Basics

# Einleitung

# REST API

# | Einleitung

REST (Representational State Transfer)

- Representational: Daten

# RESTful Principles

Die 6 Leitprinzipien für skalierbare Web-Services nach Roy Fielding

# Definition & Ursprung

- Was ist REST?

Representational State Transfer ist kein Protokoll, sondern ein **Architekturstil** für verteilte Hypermedia-Systeme. Er nutzt bestehende Web-Standards (HTTP) optimal aus.

- Die Herkunft:

Das Konzept wurde im Jahr 2000 von **Roy Fielding** in seiner Doktorarbeit vorgestellt. Es beschreibt die Kernmechanismen, die das World Wide Web skalierbar machen.

# Die 6 Prinzipien im Überblick

1. **Uniform Interface** (Einheitliche Schnittstelle)
2. **Client-Server** (Klare Trennung der Zuständigkeiten)
3. **Stateless** (Zustandslosigkeit)
4. **Cacheable** (Zwischenspeicherbarkeit)
5. **Layered System** (Mehrschichtige Architektur)
6. **Code on Demand** (Optional: Dynamische Code-Übertragung)

# | 1. Uniform Interface

Das Uniform Interface ist das zentrale REST-Prinzip: Ressourcen werden über eindeutige URIs adressiert und mit standardisierten HTTP-Methoden bearbeitet.

# REST Guiding Principles

Wir beginnen mit einem Überblick über die grundlegenden Prinzipien, die RESTful-Architekturen definieren. Diese Prinzipien, die von Roy Fielding eingeführt wurden, sind der Schlüssel zu skalierbaren und robusten Web-APIs.

## 1. Uniform Interface

- **Vorteil:** Das System wird vorhersehbar, einheitlich und extrem einfach erweiterbar.
- **Wie wird es erreicht?** Durch strikte Einhaltung standardisierter Regeln
- **RESTful Design:** Jede URI benennt **Substantive** (Ressourcen), auf die über standardisierte **HTTP-Verben** (wie `GET`, `POST`, `PUT`, `DELETE`) zugegriffen wird.

## Beispiele für das URI-Design:

- `POST /createUser`
- `GET /getUserById?id=45`
- `POST /updateUser/45`
- `GET /deleteUser/45`

**Warum falsch?** Das HTTP-Protokoll bringt die Aktions-Verben bereits mit. Zusätzliche Verben im Pfad machen die API unübersichtlich und redundant.

- `POST /users` (Nutzer erstellen)
- `GET /users/45` (Nutzer 45 abrufen)
- `PUT /users/45` (Nutzer 45 aktualisieren)
- `DELETE /users/45` (Nutzer 45 löschen)

**Warum richtig?** Die URI benennt nur das "Was" (die Ressource). Die HTTP-Methode

## URI-Struktur erklärt

Dies zeigt, wie die verschiedenen Teile einer URI zusammenarbeiten, um Ressourcen zu identifizieren und Aktionen zu steuern.

- **Ressourcenidentifizierung:** Der `path` (Pfad) identifiziert die Ressource. Zum Beispiel `/users` für die Sammlung oder `/users/45` für eine bestimmte Instanz. Hier werden **ausschließlich** Substantive verwendet.
- **Aktionssteuerung:** Die HTTP-Methode (z. B. `GET` oder `POST`, nicht Teil der URI) bestimmt die eigentliche Aktion.
- **Abfrageparameter:** Der `query` (Abfrage) wird verwendet, um Antworten zu filtern, zu sortieren oder zu paginieren (z. B. `?search=bogdan`). Dies hält die Basis-URLs sauber und übersichtlich.

## Die 4 Uniform-Constraints

- **Ressourcen-Identifizierung:** Jede Ressource ist über eine eindeutige URI ansprechbar (z.B. `GET /api/users/42`).
- **Manipulation durch Repräsentation:** Der Client besitzt nicht die echten DB-Daten, sondern modifiziert Ressourcen via Repräsentationen (JSON, XML).
- **Selbstbeschreibende Nachrichten:** Jeder Request/Response enthält alle Metadaten (Headers), die erklären, wie die Nachricht zu interpretieren ist (z.B. `Content-Type`).
- **HATEOAS:** `Hypermedia as the Engine of Application State`. Der Server liefert Links zu weiteren Aktionen mit.

## 2. Client-Server-Trennung

- **Klare Zuständigkeiten:** Das Frontend (Client) kümmert sich ausschließlich um die Benutzeroberfläche und die User Experience. Das Backend (Server) übernimmt Datenhaltung, Sicherheit und Geschäftslogik.
- **Die Stärke:**  
Beide Systeme können unabhängig voneinander weiterentwickelt, skaliert oder technologisch ausgetauscht werden.

### 3. Zustandslosigkeit (Stateless)

- **Die Kernregel:** Der Server speichert **keinen** Sitzungskontext (Session-State) über den Client zwischen zwei Anfragen.
- **Jeder Request ist autark:**  
Jede einzelne HTTP-Anfrage muss alle Informationen enthalten, die für das Verstehen und Verarbeiten notwendig sind (z.B. Authentifizierungs-Token im Header).
- **Ergebnis:** Starke Skalierung möglich:
- Ein Session-State (z. B. ein Login-Zustand) würde massiven Overhead erzeugen: Dies müsste im RAM oder in der Datenbank gespeichert werden. Wenn es gar keinen State (Zustand) gibt, kann einfach skaliert werden (= mehr Traffic auf die Webseite).
  - **Massives Load Balancing** wird möglich. Jeder Server im Cluster kann jeden Request sofort verarbeiten.

## 4. Cacheable

### Effizienz durch Caching:

Server-Antworten müssen implizit oder explizit als cachebar deklariert werden. Das spart Bandbreite, schont Server-Ressourcen und senkt die Latenz drastisch.

## 5. Layered System

### Mehrschichtiges System (Layered System):

Der Client kann nicht wissen, ob er direkt mit dem Endserver oder einem Proxy (= Stellvertreter, z. B. Load Balancer, die Traffic auf verschiedene Server verteilen) kommuniziert. Schichten verbessern die Sicherheit und die Skalierbarkeit.

## 6. Code on Demand (Optional)

- **Das Konzept:**

Der Server kann die Funktionalität des Clients temporär erweitern, indem er ausführbare Logik direkt an ihn überträgt.

- **Praxisbeispiele:**

- Senden von **JavaScript-Skripten**, die dynamisch im Browser des Nutzers ausgeführt werden.
- Historisch: Java-Applets oder Flash.

- **Hinweis:** Dies ist das einzige **optionale** Prinzip. Viele moderne APIs verzichten darauf.

## Zusammenfassung der Vorteile

| REST Prinzip      | Primärer Vorteil                                 |
|-------------------|--------------------------------------------------|
| Uniform Interface | Einfachheit, Vorhersehbarkeit & Sichtbarkeit     |
| Stateless         | Unbegrenzte horizontale Skalierbarkeit           |
| Client-Server     | Portabilität der UI & Unabhängigkeit der Systeme |
| Cacheable         | Signifikante Steigerung der Netzwerk-Performance |

# Aufgabe: RESTful URL-Design

Hier sind mehrere API-Endpunkte aus einem (fiktiven) Bibliotheksverwaltungssystem. Analysiere jede URL und erkläre, was **gegen RESTful-Design-Prinzipien verstößt** und wie man es richtig formulieren würde.

## Endpunkte

1. `GET /getAllBooks`
2. `POST /books/createNewBook`
3. `GET /books/5/delete`
4. `GET /Books/5`
5. `PUT /books/5/author/3`
6. `GET /books?id=5`

7. `DELETE /books/5/2024-01-01`
8. `POST /books/5`
9. `GET /authors/3/books/getBooksByAuthor`
10. `GET /books/active/true`

## Aufgabenstellung

Für jede URL:

- Benenne den konkreten Verstoß (z.B. Verb in URL, falsche HTTP-Methode, inkonsistente Großschreibung, falsche Hierarchie, etc.)
- Schreibe die korrigierte Version der URL inkl. der korrekten HTTP-Methode

## Bonusfrage

Bei Nummer 9 gibt es zwei mögliche "richtige" Lösungen, je nachdem, welche Ressource im Vordergrund steht. Nenne beide und erkläre den Unterschied.

# Idempotenz

*"Eine Operation ist idempotent, wenn sie mehrfach ausgeführt werden kann, ohne dass das Ergebnis über die erste Ausführung hinaus verändert wird."*

## Mathematischer Kontext

Eine Funktion  $f(x)$  ist idempotent, wenn gilt:

$$f(f(x)) = f(x)$$

Beispiele: Die Multiplikation mit 0 oder die Betragsfunktion  $abs(x)$ .

## HTTP & API Kontext

Ein Request ist idempotent, wenn der Zustand auf dem Server bei identischen Folge-Requests unverändert bleibt.

Wichtig: Der HTTP-Response-Statuscode darf sich ändern, der Zustand der Ressource hingegen nicht.

## Alltags-Analogie

### □ Der Fahrstuhl-Knopf

**Idempotent:** Egal wie oft Sie den Knopf für das 5. Stockwerk drücken: Der Fahrstuhl fährt in das 5. Stockwerk. Das Ergebnis bleibt beim 2. oder 100. Klick absolut identisch.

### Der Snack-Automat

**Nicht Idempotent:** Jedes Mal, wenn Sie den Knopf drücken (und bezahlen), fällt ein neuer Snack heraus. Ihr Kontostand sinkt weiter und der Vorrat im Automaten verringert sich.

## Warum ist das wichtig?

Gefahrlose Retries im Frontend ermöglichen.

In instabilen Netzwerken sind **Retries** (Wiederholungsanfragen) unvermeidlich.

- **Das Problem:** Ein Client sendet einen Request, der Server verarbeitet ihn erfolgreich, aber die Server-Antwort geht auf dem Rückweg im Netzwerk verloren.
- **Die Lösung:** Ist die HTTP-Methode **idempotent**, darf der Client (oder die Axios/Fetch-Logik im Frontend) den Request bei einem Timeout einfach gefahrlos noch einmal senden.
- **Kritischer Anwendungsfall:** "Bestellung abschicken" (meist **POST**) vs. "Zahlungsstatus auf bezahlt setzen" (meist **PUT**). Sie wollen niemals versehentlich Doppelabbuchungen erzeugen.

## Methoden-Vergleich im Detail

### **PUT** (Idempotent)

```
PUT /users/45
```

Wenn Sie diesen Request 10-mal hintereinander abschicken, wird der Datensatz mit der ID 45 jedes Mal mit exakt denselben Werten überschrieben. Der Endzustand der Datenbank nach dem ersten und dem zehnten Request ist absolut identisch.

### **POST** (Nicht Idempotent)

```
POST /users
```

Wenn Sie diesen Request 10-mal abschicken, erzeugen Sie im schlimmsten Fall 10 identische Nutzer mit 10 verschiedenen IDs in Ihrer Datenbank. Jede Wiederholung verändert den Zustand des Systems.

# API Dokumentation

Der Vertrag zwischen Backend, Frontend und externen Entwicklern

# Warum ist API-Dokumentation wichtig?

*"Code beschreibt, wie das System funktioniert. Dokumentation beschreibt, wie man es benutzt, ohne den Code lesen zu müssen."*

- **Das Frontend-Problem:** Ohne Dokumentation raten Frontend-Entwickler, welche Datenstrukturen (`Types`) ein Endpoint erwartet oder zurückgibt.
- **Effizienz:** Spart unzählige Slack-Nachrichten und Meetings der Marke "Wie hieß das Feld für die ID nochmal?".
- **Onboarding:** Neue Teammitglieder können sofort autark mit der API arbeiten.

# Der moderne Standard: OpenAPI & Swagger

Heute schreibt niemand mehr API-Dokumentation händisch in Word-Dateien. Man nutzt die **OpenAPI-Spezifikation (OAS)**.

- **Was ist OpenAPI?** Ein standardisiertes Format (in JSON oder YAML), das REST-APIs maschinenlesbar beschreibt.
- **Was ist Swagger?**  
Das bekannteste Tooling-Ökosystem, das aus dieser OpenAPI-Datei automatisch eine **interaktive UI** generiert.
- **Der Vorteil:** Entwickler können direkt im Browser Test-Requests an die API senden ("Try it out").

# Core-Elemente einer guten Dokumentation

Jeder dokumentierte Endpoint sollte mindestens folgende 4 Informationen liefern:

1. **Basis-Metadaten:** Die HTTP-Methode und der exakte Pfad (z.B. `POST` `/api/v1/users`).
2. **Request-Parameter:** Welche Daten müssen in den `Query-Params`, den `Headers` oder im `Body` (Payload) mitgeschickt werden?
3. **Response-Beispiele:** Wie sieht das JSON-Objekt im Erfolgsfall (`200 OK` / `210 Created`) exakt aus?
4. **Fehler-Szenarien:** Welche Statuscodes (`400`, `401`, `404`, `500`) gibt es und wie sieht die Fehlermeldung aus?

# Dokumentation-Ansätze im Vergleich

## Code-First Approach

Die Dokumentation wird **\*\*direkt im Code\*\*** generiert (z.B. über JSDoc, Decorator in NestJS oder Bibliotheken in Next.js).

✓ Bleibt aktuell, da sie nah am Code lebt.

✗ Code kann durch massig Doku-Kommentare unübersichtlich werden.

## Design-First Approach

Die API wird **\*\*zuerst\*\*** in einem Editor (z.B. Stoplight, Swagger Editor) spezifiziert, bevor Code geschrieben wird.

✓ Frontend & Backend können ab Tag 1 parallel arbeiten (Mocking).

✗ Gefahr, dass Code und Doku später auseinanderlaufen.

# Best Practices für exzellente Docs

- **Nutzen Sie semantische HTTP-Statuscodes:** Dokumentieren Sie nicht nur `200` und `500`. Zeigen Sie dem Frontend exakt, wann ein `422` (Unprocessable Entity) oder ein `409` (Conflict) verwendet wird.
- **Typsicherheit generieren:** Nutzen Sie Tools wie `openapi-typescript`. Damit lässt sich die OpenAPI-Doku einlesen, um automatisch **TypeScript-Typen fürs Frontend** zu generieren.
- **Authentifizierung klarmachen:** Es muss sofort ersichtlich sein, welche Endpoints öffentlich sind und welche ein `Authorization: Bearer <Token>` benötigen.

APIs sind Produkte. Docs die Verpackung.

Gute Dokumentation entscheidet über den Erfolg einer Architektur.

[OpenAPI](#) | [Swagger](#) | [Readme.io](#) | [Redoc](#)

# API Versionierung

Evolution ohne Frustration für Frontend und Clients

# Warum brauchen wir Versionierung?

*"Eine API ist ein verbindlicher Vertrag. Bricht das Backend den Vertrag, bricht unweigerlich das Frontend."*

Software entwickelt sich weiter, und damit auch die Datenstruktur. Wir versionieren APIs, um **Abwärtskompatibilität (Backward Compatibility)** zu garantieren.

- Wenn eine React-App Version 1 der API nutzt, darf sie nicht abstürzen, nur weil das Backend-Team an Version 2 arbeitet.
- Beide Versionen müssen für eine Übergangszeit **parallel** betrieben werden.

# Wann ist eine Versionierung nötig?

Nicht jede Änderung erfordert eine neue API-Version (z.B. `v2`). Wir unterscheiden strikt:

## ✓ Non-Breaking Change

Keine neue Version nötig.

- Hinzufügen eines **neuen** Feldes im JSON.
- Hinzufügen eines neuen Endpoints.
- Optionale Query-Parameter hinzufügen.

## ✗ Breaking Change

Neue Version (`v2`) zwingend erforderlich!

- Löschen oder Umbenennen von Feldern.
- Ändern des Datentyps (z.B. ID von `Int` auf `String`).
- Neue, verpflichtende Parameter/Payloads.

# Strategie 1: URI-Versionierung (Der Standard)

Die Versionsnummer wird direkt als Präfix in den URL-Pfad geschrieben.

- **Beispiel:** `GET https://api.example.com/v1/users`
- **Vorteile:** \* Extrem einfach zu implementieren und im Browser zu testen.
  - Für Frontend-Entwickler sofort ersichtlich, welche API genutzt wird.
- **Nachteile:**
  - Bricht streng genommen die REST-Prinzipien (Eine Version ist eigentlich keine "Ressource").

# Strategie 2 & 3: Query & Header

## Query Parameter

```
GET /users?version=1
```

Die Version wird als Filter-Parameter an die URI gehängt.

✓ URL-Pfad bleibt sauber.

✗ Verwirrend, da Query-Parameter eigentlich für Dinge wie Suchen/Sortieren gedacht sind.

## Header (Content Negotiation)

```
Accept:
```

```
application/vnd.api.v1+json
```

Der Client fordert über den HTTP-Header eine spezifische Version an.

✓ Der absolut sauberste "pure" REST-Weg.

✗ Schwer zu debuggen, da man die API nicht mehr einfach in die Browser-URL tippen kann.

# Vergleich der Strategien

| Methode                               | REST Konformität | Entwicklerfreundlichkeit | Caching im Browser |
|---------------------------------------|------------------|--------------------------|--------------------|
| URI Path<br>( <code>/v1/</code> )     | Niedrig          | Sehr hoch                | Sehr einfach       |
| Query ( <code>?v=1</code> )           | Mittel           | Hoch                     | Gut                |
| Header<br>( <code>Accept: v1</code> ) | Sehr hoch        | Niedrig (Tooling nötig)  | Komplexer          |

# Best Practices für Versionierung

1. **Halten Sie es simpel:** Nutzen Sie nur Major-Versionen (z. B. `v1`, `v2`). Ein Schema wie `v1.4.2` in der API ist Overkill und führt zu Chaos im Backend-Routing.
2. **Standard-Version definieren:** Wenn ein Client keinen Header oder Pfad angibt, leiten Sie ihn entweder zur ältesten stabilen Version weiter oder geben Sie einen sauberen Error (`400 Bad Request`) zurück.
3. **Graceful Sunset:** Wenn Sie `v1` abschalten, kündigen Sie dies rechtzeitig an (Deprecation Notices). Nutzen Sie in HTTP-Responses den standardisierten `Sunset`-Header mit einem Datum.

# Stabile Verträge. Gute APIs.

Versionierung ist das Sicherheitsnetz Ihrer Architektur.

URI Path | Query Params | Custom Headers

# Business Logic

Vom HTTP-Request zur Datenbank und zurück

# Die Evolution der Business Logic

Im Grunde genommen macht jedes Backend das Gleiche: Es nimmt einen **HTTP-Request** entgegen, jagt ihn durch die Geschäftslogik und spuckt eine **HTTP-Response** aus. Spannend wird es bei der Frage, wie wir diesen Code strukturieren, wenn die App wächst.

## Phase 1: Der Prototyp (if / else)

Man startet mit einer einzigen Datei. Die Routen und die Logik werden direkt untereinander in simplen Verzweigungen abgehandelt. Wenn die URL `/users` ist, hol den User; wenn sie `/products` ist, hol das Produkt.

**Das Problem:** Sobald Validierung, Error-Handling und Datenbanken dazukommen, bricht dieses File auseinander. Der Code wird unlesbar und lässt sich nicht mehr testen.

## Phase 2: Strukturierte Architektur

Ab hier bricht man die Riesen-Logik in saubere, wiederverwendbare Bausteine auf. Je nach Team-Präferenz nutzt man dafür einen von zwei Wegen:

- **Funktionsorientiert:** Reine, entkoppelte Funktionen (z. B. typisch für Express-Controller oder Next.js Route Handler).
- **Klassenorientiert (OOP):** Strukturierung über Controller- und Service-Klassen mit Dependency Injection (z. B. der Standard in NestJS).

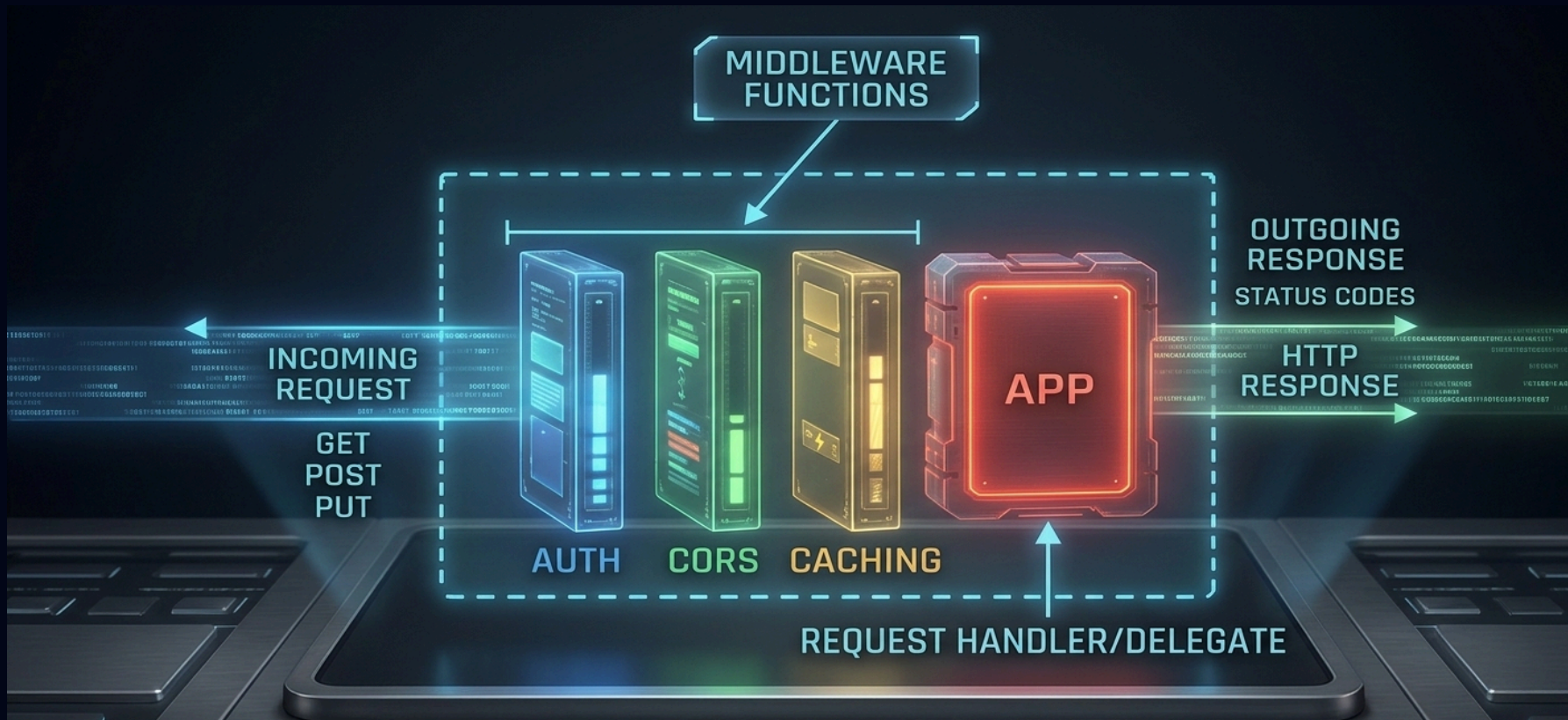
# Das Middleware & Request Handler Pattern

Um diese Struktur zu erreichen, etablieren moderne Frameworks eine saubere Kette von Verantwortlichkeiten:

- **Middleware:** Eine Funktion, die HTTP-Anfragen abfängt, **bevor** sie die eigentliche Geschäftslogik erreichen. Sie ist perfekt geeignet, um Requests zu modifizieren, Tokens zu authentifizieren oder Logs zu schreiben.
- **Request Handler:** Die finale Funktion am Ende der Kette. Sie enthält die eigentliche Geschäftslogik (z. B. den Datenbankaufruf) und sendet die fertige HTTP-Antwort (**Response**) an den Client zurück.
- **Request Handler Delegate:** Das Prinzip, diese Bausteine als Pipeline zu organisieren. Jede Middleware entscheidet aktiv, ob sie die Anfrage an den nächsten Block **delegiert** (**next()**) oder abbricht.

# Systemarchitektur: Die Pipeline

So durchläuft der Request das Backend





# Aufgabe: Der VIP-Eingang

Erstellen Sie einen simplen Express-Server mit einer geschützten Route. Ziel ist es, die Verantwortlichkeiten strikt zwischen Middleware und Handler zu trennen.

## 1. Die Middleware (Der Türsteher)

Schreiben Sie eine Middleware-Funktion `checkAuth`. Sie prüft, ob der HTTP-Header `x-role` den Wert `"vip"` hat.

- Wenn ja: Leiten Sie die Anfrage weiter.
- Wenn nein: Blockieren Sie die Anfrage mit Status `403 Forbidden` und einer Fehlermeldung.

## 2. Der Request Handler (Die Party)

Schreiben Sie eine Handler-Funktion `getVipData`. Diese wird nur aufgerufen, wenn die Middleware die Anfrage durchlässt.

- Senden Sie den Status `200 OK`.
- Schicken Sie ein JSON-Objekt zurück: `{ message: "Willkommen in der VIP-Lounge!" }`

**Route:** Verknüpfen Sie beide Funktionen auf der Route `GET /api/vip`.



# Musterlösung

So sieht die saubere Trennung im Code aus:

```
const express = require("express");
const app = express();

// 1. Middleware: Kontrolliert den Zugang
const checkAuth = (req, res, next) => {
  const role = req.headers["x-role"];

  if (role === "vip") {
    next(); // Erfolgreich: Delegiere an den nächsten Handler
  } else {
    // Abbruch: Der Request endet hier, der Handler wird nie erreicht
    res.status(403).json({ error: "Zugriff verweigert. Nur für VIPs!" });
  }
};

// 2. Request Handler: Kümmert sich nur um die Business Logik
const getVipData = (req, res) => {
  res.status(200).json({ message: "Willkommen in der VIP-Lounge!" });
};

// Verknüpfung der Pipeline
```

# API Architektur & Modellierung

Struktur, Verhalten und Implementierungs-Stile

# Das "Duo" der Backend-Architektur

Um eine professionelle REST-API zu bauen, trennen wir zwischen der **technischen Struktur** und dem **logischen Ablauf**:

## 1. Die Struktur

Das **Middleware & Request Handler Pattern** bildet das Skelett der API. Es definiert, wie ein Request die technische Pipeline (Auth, Validierung, Logging) durchläuft, bevor er die Logik erreicht.

## 2. Das Verhalten

**Sequenzdiagramme** modellieren die eigentliche Business Logic. Sie zeigen visuell, wie Daten zwischen Middleware, Handlern, Services und Datenbanken fließen.

# Modellierung mit Sequenzdiagrammen

Warum sind Sequenzdiagramme das perfekte Werkzeug für das Middleware-Pattern? Weil HTTP-Anfragen von Natur aus **sequentiell** ablaufen.

*Das Diagramm zeigt nicht "Wie wird sortiert?", sondern "**Wer spricht wann mit wem?**"*

- **Visualisierung der Kette:** Man sieht sofort, wo eine Middleware den Request abbricht (z. B. 401 Unauthorized).
- **Zuständigkeiten:** Es wird klar, welche Komponente für die Datenvalidierung zuständig ist und welche für den Datenbank-Query.
- **Schnittstellen:** Die Pfeile im Diagramm definieren die Methoden-Aufrufe oder Funktions-Parameter.

# Architekturstile: FP vs. OOP

Beide Patterns (Middleware & Sequenz-Modellierung) funktionieren unabhängig vom gewählten Programmierstil. Der Unterschied liegt nur in der **Art der Bausteine**.

## Funktionsorientiert (FP)

Typisch für Express.js.

- Bausteine im Diagramm sind **reine Funktionen**.
- Der Request-Status wird als Datenobjekt durch eine Kette (Pipe) gereicht.
- `req -> func1 -> func2 -> res`

## Objektorientiert (OOP)

Standard in NestJS / Spring Boot.

- Bausteine im Diagramm sind **Instanzen von Klassen** (Objekte).
- Der Request triggert Methoden auf Objekten (Controller, Services, Guards).
- `ObjectA.method() -> ObjectB.call()`

# Zusammenfassung

1. Das **Middleware-Pattern** ist das infrastrukturelle Gerüst (Die Pipeline).
2. Das **Sequenzdiagramm** ist der Bauplan für den Datenfluss (Die Logik).
3. **FP** oder **OOP** sind lediglich die Werkzeuge, mit denen wir diese Architektur im Code umsetzen.

**Egal welcher Stil:** Das Ziel ist die Entkopplung von technischer Infrastruktur und fachlicher Business Logic.

# Databases Fundamentals

Die Grundlage für persistente Daten und stabile Geschäftsdomänen

# SQL vs. NoSQL: Die zwei Welten

Datenbanken werden grundlegend in **relationale (SQL)** und **nicht-tabellarische (NoSQL)** Systeme unterteilt. Im echten Entwickleralltag überwiegt meist der relationale Ansatz, da sich Business-Logiken hervorragend über feste Beziehungen abbilden lassen.

## SQL (Relational)

- Basiert auf strikten **Beziehungen**
- Festes, vordefiniertes **Schema**
- Geringere Roh-Performance
- Schwieriger horizontal zu skalieren
- **Tech-Stack:** PostgreSQL, MySQL

## NoSQL (Non-Tabular)

- Keine starre Form (Schemaslos)
- Eingeschränkte Beziehungslogik
- Sehr hohe Performance
- Hervorragend horizontal skalierbar
- **Tech-Stack:** MongoDB, Neo4j

# Wann nutzt man was?

Die Wahl des Datenbanksystems hängt von den Anforderungen der Anwendung ab:

| Kriterium | SQL-Datenbanken                   | NoSQL-Datenbanken                     |
|-----------|-----------------------------------|---------------------------------------|
| Fokus     | Komplexe Beziehungen wichtig      | Maximale Performance & Durchsatz      |
| Schema    | Benötigt ein verlässliches Schema | Flexibel für unstrukturierte Daten    |
| Beispiele | E-Commerce-Systeme, Banking       | Content-Management (CMS), Log-Storage |

# Query (Abfrage) Basics

Wie wir mit Datenbanken kommunizieren: SQL vs. NoSQL

# Die zwei Sprachen der Datenbeschaffung

Egal welche Architektur: Im Kern geht es immer um **CRUD** (Create, Read, Update, Delete). Aber **wie** wir diese Anfragen (Queries) formulieren, unterscheidet sich fundamental:

## Der SQL-Ansatz

**Deklarativ:** Sie beschreiben exakt, **was** Sie haben wollen, aber nicht, wie die Datenbank es intern beschafft. Die Syntax ist eine eigene, englisch-ähnliche Sprache (Structured Query Language).

## Der NoSQL-Ansatz

**Prozedural / Objektbasiert:** (Am Beispiel MongoDB). Sie rufen Methoden auf Collections auf und übergeben Filterbedingungen als JSON-Objekte. Das fühlt sich für



# SQL Query Basics

Die Anatomie einer Standard-SQL-Abfrage besteht fast immer aus denselben Klauseln:

- **SELECT**: Bestimmt, **welche Spalten** zurückgegeben werden.
- **FROM**: Bestimmt, aus **welcher Tabelle** gelesen wird.
- **WHERE**: Die **Filterbedingung** (z.B. ID, Status, Datum).
- **JOIN**: Verknüpft Datensätze aus **zwei Tabellen** anhand einer gemeinsamen ID.

```
-- Hole Namen und Email aller aktiven User über 18
SELECT first_name, email
FROM users
WHERE age > 18 AND status = 'active';
```



# NoSQL Query Basics (MongoDB)

In MongoDB arbeiten wir nicht mit Tabellen und Zeilen, sondern mit **Collections** und **Dokumenten** (im Grunde JSON-Objekte).

- `db.collection.find()`: Die Standardmethode zum Lesen.
- **Filter-Objekt**: Das erste Argument ist ein Objekt mit den Filter-Bedingungen.
- **Operatoren**: Statt `>`, `<` oder `=` nutzen wir spezielle Keys wie `$gt` (greater than) oder `$eq` (equal).

```
// Exakt gleiche Logik wie die SQL-Query zuvor
db.users.find(
  { age: { $gt: 18 }, status: "active" }, // Filter (WHERE)
  { first_name: 1, email: 1, _id: 0 }, // Projection (SELECT)
);
```



# CRUD: Der direkte Syntax-Vergleich

| Operation | SQL (Relational)                                            | MongoDB (NoSQL)                                                      |
|-----------|-------------------------------------------------------------|----------------------------------------------------------------------|
| CREATE    | <pre>INSERT INTO<br/>users (name)<br/>VALUES ('Tom');</pre> | <pre>db.users.insertOne({ name:<br/>'Tom' })</pre>                   |
| READ      | <pre>SELECT * FROM<br/>users;</pre>                         | <pre>db.users.find({})</pre>                                         |
| UPDATE    | <pre>UPDATE users<br/>SET age = 20<br/>WHERE id = 1;</pre>  | <pre>db.users.updateOne({ _id: 1<br/>, { \$set: { age: 20 } })</pre> |

# Das "Join" Problem

Der größte funktionale Unterschied beim Schreiben von Queries:

- **SQL ist fürs Verknüpfen gebaut:** Ein `INNER JOIN` in SQL ist hochoptimiert und der absolute Standardweg, um z. B. einen User mit seinen Bestellungen zu verknüpfen.
- **NoSQL speichert Daten zusammen:** In MongoDB gibt es zwar `$lookup` (eine Art Join), aber eigentlich speichert man Daten, die oft zusammen abgerufen werden, direkt in einem einzigen verschachtelten Dokument. Man modelliert die Datenbank nach den Queries, nicht nach Beziehungen.

Daten abfragen ist **Logik**.

| Das Tool entscheidet nur über die Syntax.



# Aufgabe:

- **SQL Practice:**
  - Sie müssen SQL-Queries nicht zwingend auswendig im Schlaf schreiben können, aber das Verständnis für relationale Operationen ist Pflicht.
  - **Fokus im Interview:** Beherrschen Sie `INNER JOIN` und `LEFT JOIN` sicher auf einfachem bis mittlerem Schwierigkeitsgrad.
- **NoSQL (MongoDB) Practice:**
  - Machen Sie sich mit dokumentenorientierten Abfragen vertraut.
  - Nutzen Sie Tools wie `humongous.io`, um grundlegende CRUD-Queries gegen eine Live-NoSQL-Datenbank auszuführen und sich auf typische Screening-Fragen vorzubereiten.

# Das SQL-Datenbankschema

In der relationalen Welt ist das **Schema** das Fundament der Datenintegrität. Es beschreibt exakt die Struktur und die Datentypen aller Entitäten.

*"Ein Schema macht Datenbankabfragen verlässlich, typsicher und vorhersagbar."*

- **Sicherer Vertrag:** Änderungen am Datenmodell erfordern explizite Migrationen (Hinzufügen von Tabellen, Spalten oder neuen Fremdschlüssel-Beziehungen).
- **Visuelle Modellierung:** Beziehungen zwischen Entitäten (z. B. wie ein `actor` mit `movies` verknüpft ist oder wo `movie_reviews` andocken) werden in Entity-Relationship-Diagrammen (ERD) strukturiert.

# Daten halten. Struktur bewahren.

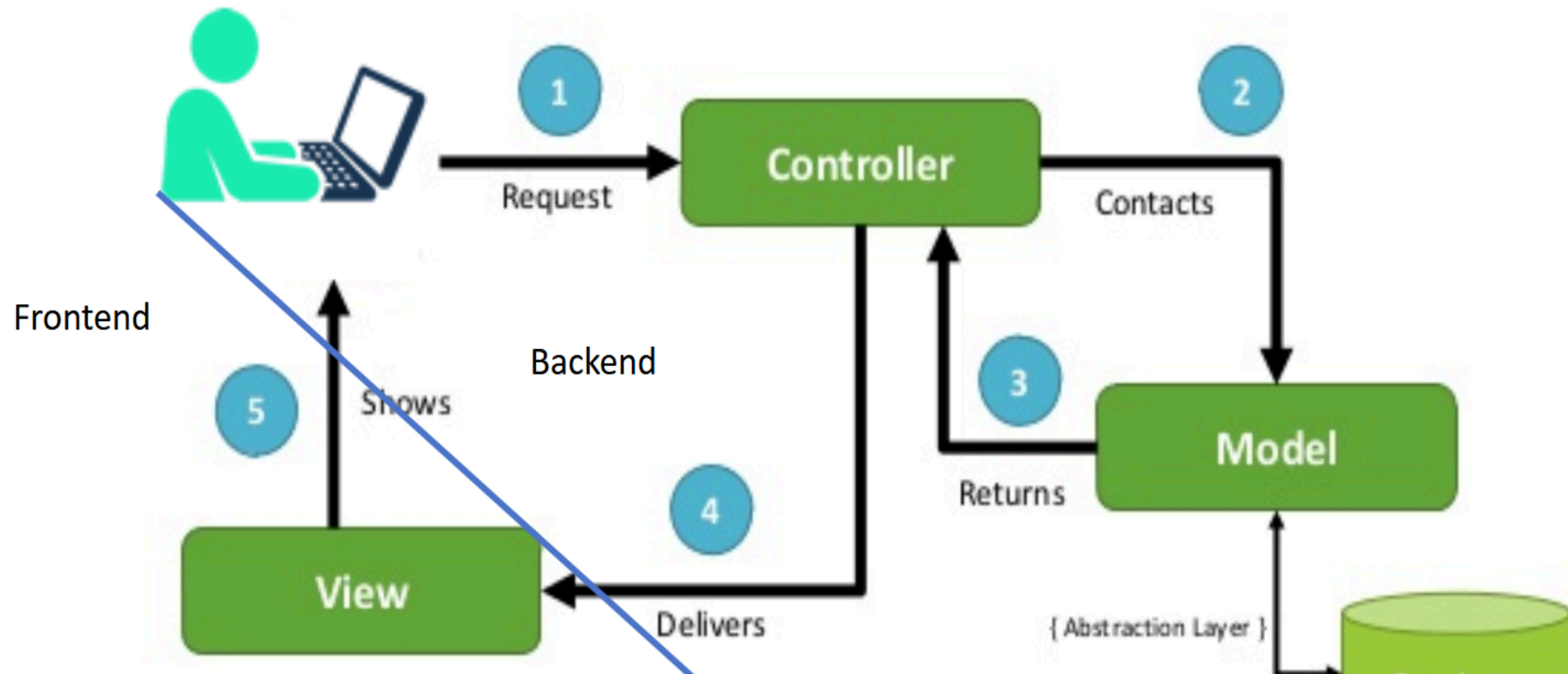
Die Wahl der Datenbank bestimmt das Schicksal der Skalierung.

PostgreSQL | MongoDB | ERD | Schema Design

| ORM

# Wrap it up

## MVC



- Model: Repräsentation der Daten
- View: Darstellung der Daten für den Benutzer
- Controller: Vermittler zwischen Model und View, verarbeitet Benutzereingaben; Geschäftslogik
- Früher war View ein HTML-Template, heute ist es oft die REST API. Zusätzlich gibt es dann eine Single Page Application (SPA) als Frontend für Nutzer.
  - zum Beispiel ASP.NET MVC, Spring MVC, Ruby on Rails